Front End:

CITY INFRUSTRUCTURE PROJECT REQUIREMENTS DOCUMENT

Project update

Front End

Languages: CSS, JS, HTML

Progress:

- Attempted to make styling more appealing and consistent. Made the color palette consistent through all pages, and fixed some cases of off colors.
- Begun work on adding a proper profile page.
- Added cancel button to settings page.
- Started working on integration with the backend branch.
- Begun adding Java files for every page.
- Added error and confirmation messages for some pages.
- Also began to discuss branch restructuring.

Issues:

- Login form submits without feedback or validation.
- Issues with the admin page (only with certain users??)
- Fix issues relating to some users having missing files and a duplicate set of files

Next Sprint Tasks

- Finish adding Java files for every page.
- Create separate packages for css, html and js files.
- Work on servlets

QA

Quality Assurance Team

(Roi, Hector)

1. Questions/Concerns to bring up:

- Has a unit testing framework already been created? Is that what Maven was for? I remember we had j unit do we still have this?
- We talked about Maven in class. Are we deciding not to use it?
- Only admin page has errors logged in console, would be great if all other html pages had this to ensure user knows when something does or doesn't work.
- We want this stuff pushed to main quickly according to wolfe i think, yet we are behind due to a lot of missing functionality.
- What's the automated testing framework in github action javier was talking about?
- Do we add the undone things to git issues to inform everyone, or do we tell javier and he does this?

2. Quality of pages

HTML Pages:
Hamburger menu works on all pages.(3 vertical dashes)
After booting up the docker version, SMALL bug is that whenever user clicks home tab it shows the index.html for a second then brings user to login page;
No issues found in the consoles, except for admin page(errors indicate css issues and no backend connection):
“admin-view.html:1 Refused to apply style from 'http://127.0.0.1:5501/src/main/webapp/admin-view.css' because its MIME type ('text/html') is not a supported stylesheet MIME type, and strict MIME checking is enabled. :5501/api/admin/departments:1 Failed to load resource: the server responded with a status of 404 (Not Found)
:5501/api/admin/reports:1 Failed to load resource: the server responded with a status of 404 (Not Found)
admin-view.html:1 Refused to apply style from 'http://127.0.0.1:5501/src/main/webapp/admin-view.css' because its MIME type ('text/html') is not a supported stylesheet MIME type, and strict MIME checking is enabled.

Index.html
Nothing appears on the page except the basic startup black page saying “DevOps baseline is running.”
Login-page.html
Sign Up button does not work; it should take the user to the register page.
There is a map on this page, but other pages do not have one. Maybe this one can be reused.
When signing in with a fake user and password, the page refreshes, which suggests the information might be submitted, but no error message appears telling the user something like “We could not find you in our records.”
Profile button doesn’t redirect to a user profile page
Map overlays hamburger menu after it’s opened

Login page and home page are identical - is this intentional?
Dashboard-page.html
No buttons here to test. Reports and updates tabs correctly redirect.
Admin-view.html
No data here to load in and test.
There is an error message here (the only one found on the front end), so take note of how it was made to create others.
Admin profile button doesn't redirect to an admin profile page
"Save Updates" button does nothing
Status dropdown and budget input works
Submission-page.html
Nothing happens when pressing "Submit a Report."
Map does not display anything.
No error messages appear on this page when things fail.
File upload opens successfully.
Report-tracking-page.html
There are fake reports here, but fake report data is not consistent across the other pages.
Updates-page.html
No issues noted. Displays placeholder updates with issue names, statuses, and dates.
UserRegistration.html
When entering sign-up information, there is no indication that anything was saved.
When entering passwords that do not meet the requirements, no error message appears.
Frontend and Integration Issues:

login-page.html
Console error: login-page.css failed to load (404 Not Found)
submission-page.html
Console error: submission-page.css failed to load (404 Not Found)

admin-view.html
Console error: admin-view.css failed to laid (404 Not Found)
Backend endpoints not implemented or incorrect routes
Frontend attempting API calls receive no valid response

Direct API Testing:
http://localhost:8080/health
Black screen - "OK"
http://localhost:8080/db-check
Black screen - "DB OK"

Overall:
Missing static resources:
Multiple pages fail to load css files (404 errors)
Broken or inconsistent UI
Broken frontend-backend integration
API calls return 404
Admin functionality cannot load data
No network activity on user actions
Buttons do not trigger API calls
Core features (login, submit, sign up) not implemented or not wired

3. Protocols for future QA:

- There is a folder named QA Scripts; currently only one script file is there, and one line in the readme has the command to run that script. There is a .txt of the latest run showing the tests were passed. The script tested the API calls. Make more scripts for automated testing if teams need it.
- Update and change documentation as needed
- We suggested a color scheme for this document: highlighting sentences RED once these issues have been added to git issues, waiting for them to be resolved; GREEN once they are resolved.
- Currently as of 3/26/2026 our updates are limited due to integration still not being in place, so we are not able to mark anything as completed
- We've put emphasis on having error messages, but still lack good logging from teams so we would love to push this going forward!

Backend

Backend Documentation (R3)

(Ava, Ethan, Jana, Madeline, Ellie)

- **ReportUpdate (Madeline)**
  - Filled ReportUpdateRepository.java
  - Moved ReportUpdate (model) file to the correct folder
  - Moved ReportUpdateDTO to the correct folder
  - Corrected capitalization errors
  - **PURPOSE:** ReportUpdateDTO and ReportUpdate (model) had miscapitalized classes causing errors, as well as being in the wrong folder. These were moved to the correct placement.
  - **PURPOSE:** the Repository Java file handles direct database queries to update different parts of the submitted report. It is only called by ReportService. The file uses 'DepartmentRepository.java' as an example for how it should be structured.
- **Webapp folder (Madeline)**
  - Deleted stray draft .css files that were already in the 'Infrastructure-Reporter.css' file for clarity and to prevent future errors
  - **PURPOSE:** there were miscellaneous .css files labeled as drafts left over in the files from the first sprint round. I knew I already included them in 'Infrastructure-Reporter.css' before I left that round, and that there had been a request to delete them but no one ever did, so I took matters into my own hands.
- **pom.xml (Madeline)**
  - Removed json dependency
    - This can be added back later if an issue pops up; however, Docker should be able to handle any json files without a dependency
  - **PURPOSE:** there wasn't a reason to have it, nothing used it
  - **FUTURE:** Depending on how things turn out with maven/building, it may be necessary to remove the pom entirely (this is a LATER DECISION) (due to Wolfe saying things should be built through shell instead but he's ok if we finally decide on Maven
- **ReportServiceTest (Madeline)**
  - Filled ReportServiceTest.java
  - Modified ReportController.java
  - Modified ReportService.java
  - **PURPOSE:** ReportServiceTest is a dummy test thrown to test the report service and return the correct messages according to conditions outlined in the file. It uses the DepartmentServiceTest.java file as an example for code format
  - **PURPOSE:** the other two were just tweaked slightly to have the correct methods called

---

- **DepartmentController (Jana)**
  - Added logging statements for all GET endpoints
  - Logged incoming requests and endpoints hits
  - Logged success responses including returned data counts
  - Logged error cases (invalid ID, not found, internal errors)

    - **PURPOSE:** Logging was added for debugging purposes. This allows everyone to be able to trace API usage, monitor failures, and verify correct endpoint behavior. (Also for professor wolfe :D).
- **ImageController (Jana)**
    - Changed @WebServlet mapping from /api/reports/* to apo/image/*
    - Updated path matching logic to reflect new route structure
    - Added logging statements across POST and GET endpoints
    - Logged authentication failures (missing/invalid JWT, forbidden roles)
    - Logged upload success with image metadata
    - Logged retrieval success for report images
    - **PURPOSE:** The servlet mapping change was resolved because there was conflict in Tomcat where two controllers shared the same route. Logging additions improve traceability for uploads and authentication issues.
- **AuthController (Jana)**
    - Added logging statements to all authentication endpoints
    - Logged requests hit for /register, /login, and /me
    - Logged successful registration and login attempts
    - Logged failed authentication attempts with reasons
    - Logged JWT validation failures
    - **PURPOSE:** Logging was added for debugging purposes. This allows everyone to be able to trace API usage, monitor failures, failed login attempts, and token validation issues.
- **ReportController (Jana)**
    - Updated servlet routing to properly support:
        - /api/reports/{id}
        - /api/reports/{id}/updates
    - Added new GET endpoints
        - Retrieve single report
        - Retrieve report updates
        - Retrieve current user's reports (/api/reports/my)
    - Fixed createdBy assignment
        - Removed hardcoded value
        - Dynamically have it read userId from JWT filter
    - Added PUT endpoint
        - /api/reports/{id}/status (Admin only)
    - Added DELETE endpoint
        - /api/reports/{id} (Admin only)
    - Added logging statements across all endpoints
    - Logged success and failure cases for all operations
    - **PURPOSE:** Added functionality where the reports can be created, read, updated, status, and deleted by admin. Logging added for debugging and monitoring.
- **ImageStorageService (Jana)**
    - Fixed missing reportId assignment
        - Added reportImage.setReportId(reportID)

    - **PURPOSE:** This fix resolved a database integrity issue where images were being saved with a report_id of 0, which violated the foreign key constraints. This change ensures proper linkage between reports and images.
- **ReportService (Jana)**
    - Added ReportUpdateRepository as dependency
    - Added new methods:
        - getReportById(long id)
        - getUpdatesByReportId(long reportId)
        - updateStatus()
        - deleteReport(long reportId)
        - getMyReports(long userId)
    - **PURPOSE:** These changes allowed for report retrieval by ID, viewing report update history, updating report status with validation, deleting reports, fetching user-specific reports.
- **ReportUpdateRepository (Jana)**
    - Fixed department_id:
        - Uses setNull() when departmentID == 0 instead of inserting 0
    - **PURPOSE:** Prevents foreign key constraint violations by ensuring that invalid department IDs are stored as NULL rather than 0.
- **JwtAuthFiler (Jana)**
    - Added @WebFilter("/api/*") annotation
    - Fixed user ID extraction:
        - Changed from claims.get("userId") to Long.parseLong(claims.getSubject())
    - **PURPOSE:** The filter annotation ensures the authentication layer is actually applied to incoming requisitions. Fixing user ID extraction ensures JWT claims are correctly parsed, enabling proper authentication and authorization across the backend.

- **ImageStorageServiceTest (Ava)**
    - Created to test Image Storage logic
    - Included error logs
    -
    - PURPOSE: To cover image upload, retrieval, deletion
- **AuthServiceRepository (Ava)**
    - Created to

- **AuthService / AuthServiceTest (Ethan)**
    - **Updated AuthService.java to match the repository changes where user lookup methods now return Optional instead of direct objects**
    - **Changed old null-based checks to use isPresent() for username and email/phone validation**
    - **Added and expanded AuthServiceTest.java to cover registration, login flow, validation errors, and edge cases using test data instead of relying on a real database**

    - **PURPOSE: These changes made the authentication service match the updated backend repository structure and improved backend testing coverage for signup and login behavior. This also helped catch validation issues earlier without needing the full database running every time.**
- **seed.sql (Ethan)**
    - **Fixed database seed data so generated sample records matched valid ID ranges already present in the database**
    - **Adjusted the sample insert logic to prevent startup and initialization issues during backend launch**
    - **PURPOSE: The seed data was causing backend launch problems because generated records were not lining up correctly with valid existing data. Fixing this made database initialization more reliable and made the sample backend data usable for testing.**
- **Backend launch / Docker backend setup (Ethan)**
    - **Worked through backend launch issues related to the current Docker setup and database initialization**
    - **Cleaned up backend-side configuration problems that were interfering with startup and testing**
    - **PURPOSE: This work helped the backend start more consistently and made it easier to test backend features without the project failing during initialization.**
- **ReportRepository (Ethan)**
    - **Worked on ReportRepository.java backend functionality as part of the repository layer for 'report' database access**
    - **Helped build out repository backend support needed for report operations**
    - **PURPOSE: ReportRepository.java is part of the backend data access layer and is needed so report-related backend features can properly communicate with the database.**
- **ReportUpdate / DTO integration (Ethan)**
    - **Helped push Anderson's ReportUpdate.java and ReportUpdateDTO.java work into the backend codebase so it could be included with the current backend progress**
    - **PURPOSE: This helped make sure report update backend code was integrated into the project and available for use with the rest of the backend structure.**

- **Seed.sql (Ellie)**
    - Configured series to easily generate sample backend data for both users and reports for quick initialization and testing
    - Tied the generated reports to the user accounts
    - Fixed typos and syntax errors
    - **PURPOSE**: This automated the seeding process and allowed more robust database and records testing to be done by removing the need to hardcode users and reports manually
- **userService.java (Ellie)**
    - Edited file to properly handle the logic of the user functionalities within the methods used for updating and deleting profiles
    - Fixed the exceptions so they were no longer generic

    - Wrote method validateUserId() to simplify validation instead of repeating it
    - Enforced the front-end security testing at user registration with email and password verifications
    - **PURPOSE**: The changes made cleaned up the code and enforced the security implemented in the front-end
- **Pom.xml (Ellie)**
    - Modified the file to include the Spring framework dependencies used in the userService.java file
    - **PURPOSE**: Used to support the userService file

API Breakdown (Jana) This will show all the API's that are being used in the project.

Auth Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user
POST	/api/auth/login	No	Log in — returns JWT token
GET	/api/auth/me	Yes (any role)	Get current user info from token

Report Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/reports	No	Get all reports
GET	/api/reports/{id}	No	Get single report by ID
GET	/api/reports/{id}/updates	No	Get full status history for a report
GET	/api/reports/my	Yes (Citizen or Admin)	Get current user's reports
POST	/api/reports	Yes (Citizen or Admin)	Submit a new report
PUT	/api/reports/{id}/status	Yes (Admin only)	Update report status
DELETE	/api/reports/{id}	Yes (Admin only)	Delete a report (cascades to updates + images)

Image Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/images/{reportId}	Yes (Citizen or Admin)	Upload an image to a report
GET	/api/images/report/{reportId}	No	Get all images for a report

Department Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/departments	No	Get all departments
GET	/api/departments/{id}	No	Get single department
GET	/api/departments/{id}/contacts	No	Get contacts for a department

Devops

DEVOPPS TEAM
(Brody, Javier, Sophina, Carter)

BRODY:
Overview

Over the past two weeks, I focused on containerizing the application using Docker. The project initially explored a multi-container setup (including a frontend nginx container), but was simplified to a two-container system (Tomcat backend + PostgreSQL database) to reduce complexity and match requirements.

Frontend Container
A separate nginx container was briefly used to serve static frontend files and proxy API requests to the backend. While functional, it added unnecessary complexity and was removed in favor of using Tomcat to serve both frontend and backend.

Final Architecture
Browser → Tomcat → PostgreSQL

Containers

Backend/API (Tomcat)
- Runs Java servlets
- Serves frontend from:
  ```
  src/main/webapp/
  ```
Database (PostgreSQL)
- Stores application data
- Initialized with SQL scripts

Ports

- App: `localhost:5050 → container:8080`
- Database: `localhost:5433 → container:5432`

How the System Works

```
User → Tomcat (servlets)
    → JDBC
    → PostgreSQL
    → Response
```

```

## Technologies Used

- Java (Servlets)
- Apache Tomcat 10+
- Maven
- Docker & Docker Compose
- PostgreSQL
- JDBC

## Key Contributions

- Built and debugged container setup
- Simplified architecture to 2 containers
- Integrated frontend into Tomcat
- Verified full system runs end-to-end

## How to Run

```bash
docker compose down -v --remove-orphans
docker compose up --build
```
Then open:

```
http://localhost:5050
```

Notes

- Frontend should be edited in:
  ```
  src/main/webapp/
  ```
- Do not use old frontend container setup
- Tomcat handles both frontend and backend
  ```

CARTER:

Backend API & Database Containers

The dockerfile in this folder is connected to the container responsible for linking together the frontend services & the other container's content, the PostgreSQL database & data.

API

The Backend API dockerfile is the exact same structure as that of the primary "tomcat" container used in the main directory, though since the dockerfile has to be in a folder to be recognized as different than the main tomcat dockerfile, it has to reference a parent directory, which is why the code in the dockerfile & docker-compose is slightly different. The Backend API container must rely upon the .env file in the parent directory to connect to the database in the database container, and must expose port 8080 and transfer to URL as port 5050; this means the URL to access is 5050. The backend API container is dependent upon the health of the database container. The backend API is referenced to connect frontend through the proxy passes in the nginx.conf file in the Frontend directory; db_check and api servlets referred to in the Frontend are proxy-passed to the API container, which can properly connect to the database where Frontend cannot.

DB

The database container is fully instantiated & handled through the docker-compose.yml in the parent directory, where a volume "db_data" is created and maintained to persistently store the database contents, and ./DB/init:/docker-entrypoint-initdb.d is used as a volume mount command to use the files in the DB/init folder of the repository to instantiate and generate all the SQL in those files on the first container use. The database container needs to connect to the port hosting the database, which is currently 5433, through the common PostgreSQL user port, 5432; the docker-compose is currently dynamically setting this up so that, if the host-port is changed, the port used in the command to connect the ports is also automatically changed.

Requirements

Requirements Team

(Ameen, Makayla)

Created Requirements Documents:

- Continuous Integration Requirements Document
- Tech Stack Graphic With descriptions
- City infrastructure Team Update Document
- City Infrastructure Web Pages Requirements Documents
- Java Servlet List